



Escuela de Informática y Telecomunicaciones

Analista Programador

Taller de Programación Aplicada

FarmaCode

Comparador de Fármacos Bioequivalentes

Informe Técnico de Proyecto

Integrantes

Antonio Chihuailaf

Iván Carrasco

Matías Bórquez

Sergio Puebla

Profesor Guía: Félix Cifuentes

0 Contents

1	Introducción	3
2	Contexto del Proyecto	4
3	Metodología de Desarrollo	5
3.1	Scrum como Marco Ágil	5
3.2	Roles del Equipo	5
3.3	Herramientas de Gestión	6
4	Patrones de Diseño	7
4.1	MVVM en el Frontend Android	7
4.2	Microservicios en el Backend	7
4.3	Repository Pattern	7
5	Arquitectura del Sistema	8
5.1	Visión General	8
5.2	Arquitectura del Frontend (Android – MVVM)	8
5.3	Arquitectura del Backend (Microservicios – Spring Boot)	8
5.4	Flujo de una Consulta Típica	8
6	Lenguaje de Programación y Stack Tecnológico	10
6.1	Tabla de Tecnologías	10
6.2	Justificación de las Tecnologías Clave	10
6.2.1	Kotlin y Jetpack Compose	10
6.2.2	Java con Spring Boot	11
7	Configuración del Servidor	12
7.1	Railway como Plataforma Cloud	12
7.2	Configuración de Cada Microservicio en Railway	12
7.3	Modelo de Servicios Cloud	12
8	Aspectos de Integración	14

8.1	Integración OCR con ML Kit	14
8.2	Integración con la API de Gemini	14
8.3	Integración Android ↔ Backend (Retrofit)	15
9	Conceptualización de la Solución	16
9.1	Atributos de Calidad	16
11	Conclusión	17
10	Mockup	18

1 Introducción

El presente informe técnico describe en detalle el diseño, la arquitectura y el proceso de desarrollo del proyecto **FarmaCode**, una aplicación móvil orientada a empoderar al ciudadano chileno en el acto de compra de medicamentos. La aplicación permite comparar fármacos de marca con sus equivalentes bioequivalentes, entregando información de costo y composición en tiempo real mediante el uso de tecnologías modernas de visión computacional e inteligencia artificial.

El documento se organiza en torno a los aspectos fundamentales del proyecto: la metodología de desarrollo adoptada, los patrones de diseño aplicados, la arquitectura del sistema, el stack tecnológico seleccionado, la configuración del servidor y las integraciones externas implementadas. Cada sección busca dar cuenta de las decisiones técnicas y su justificación en el contexto del proyecto.

2 Contexto del Proyecto

FarmaCode surge como respuesta a una problemática estructural del mercado farmacéutico chileno: la asimetría de información entre el consumidor y el proveedor en el punto de venta. El sistema actúa como un nodo de información que optimiza la toma de decisiones del usuario mediante el acceso inmediato a datos farmacológicos críticos y comparativas de mercado en tiempo real.

El flujo operativo del sistema se compone de los siguientes hitos:

- **Digitalización de insumos:** captura automatizada mediante OCR o ingreso manual.
- **Procesamiento inteligente:** identificación del principio activo mediante modelos de lenguaje natural (Gemini API).
- **Análisis comparativo:** cruce con bases de datos de bioequivalentes validados por el ISP.
- **Visualización estratégica:** presentación de alternativas jerarquizadas por eficiencia de costos.

Este modelo reduce la dependencia del usuario respecto al vendedor y mejora la transparencia del proceso de compra, con especial impacto en sectores de menores ingresos y enfermos crónicos que requieren medicación continua.

3 Metodología de Desarrollo

3.1 Scrum como Marco Ágil

El equipo adoptó Scrum como metodología de desarrollo, dado su carácter iterativo e incremental, que resulta especialmente adecuado para proyectos donde los requerimientos pueden evolucionar durante el ciclo de vida. Scrum permite entregar valor de forma continua al tiempo que mantiene la capacidad de adaptarse a cambios de alcance o prioridad.

Los pilares de la implementación de Scrum en FarmaCode son:

- **Sprints de dos semanas:** cada iteración incluye planificación, desarrollo, revisión y retrospectiva.
- **Product Backlog:** lista priorizada de historias de usuario que guía el trabajo del equipo.
- **Sprint Backlog:** subconjunto de tareas comprometidas para el sprint en curso.
- **Daily Standup:** reunión diaria de sincronización para identificar bloqueos y alinear avances.
- **Sprint Review:** demostración de funcionalidades completadas al término de cada sprint.
- **Retrospectiva:** análisis de proceso para mejora continua del equipo.

3.2 Roles del Equipo

La distribución de roles dentro del marco Scrum quedó definida de la siguiente manera:

Rol	Responsabilidad	Integrante(s)
Scrum Master / Líder	Coordinación, gestión de impedimentos y facilitación de ceremonias	Antonio Chihuailaf
Backend Developer	Lógica de negocio, API REST, microservicios, base de datos	Iván Carrasco
Android Developer	Aplicación móvil Kotlin, Jetpack Compose, integración OCR	Matías Bórquez
QA / Frontend Support	Pruebas, documentación y soporte de desarrollo	Sergio Puebla

Table 1: Roles y responsabilidades del equipo de proyecto.

3.3 Herramientas de Gestión

- **Control de versiones:** Git con repositorios en GitHub, bajo la organización del equipo.
- **Gestión de tareas:** tablero Kanban organizado en columnas *Backlog*, *In Progress*, *Review* y *Done*.
- **Comunicación:** canales de mensajería para sincronización asíncrona entre sprints.
- **Documentación:** generada de forma continua y adjunta a cada entregable de sprint.

4 Patrones de Diseño

4.1 MVVM en el Frontend Android

La capa de presentación de la aplicación Android sigue el patrón **Model-View-ViewModel (MVVM)**, recomendado por Google para el desarrollo moderno con Jetpack. MVVM establece una separación clara entre la lógica de presentación y la interfaz de usuario, facilitando la testabilidad y el mantenimiento del código.

Los roles dentro del patrón son:

- **Model:** entidades de dominio y repositorios de datos (Room, API REST).
- **ViewModel:** contiene la lógica de presentación, gestiona el estado de la UI mediante `StateFlow` y `LiveData`, y expone funciones de acción.
- **View:** pantallas Jetpack Compose que observan el estado del ViewModel y recomponen la UI de forma reactiva.

Este enfoque permite que la UI sea completamente declarativa y reactiva al estado, eliminando el acoplamiento entre capas y simplificando la gestión del ciclo de vida de los componentes.

4.2 Microservicios en el Backend

El backend adopta una arquitectura de microservicios, donde cada servicio es una unidad autónoma responsable de un dominio de negocio específico. Los servicios se comunican a través de APIs REST con formato JSON, siguiendo los principios de bajo acoplamiento y alta cohesión.

Los microservicios identificados en FarmaCode son:

- **Servicio de Medicamentos:** gestiona el catálogo farmacológico, principios activos y equivalencias.
- **Servicio de Usuarios:** autenticación, autorización y gestión de perfiles.
- **Servicio de Consultas:** procesamiento de solicitudes OCR y consultas a la API de Gemini.
- **Servicio de Precios:** comparación de precios por farmacia y zona geográfica.

4.3 Repository Pattern

Tanto en Android como en el backend se aplica el **patrón Repository**, que actúa como capa de abstracción entre el origen de los datos (base de datos local, API remota) y la lógica de negocio. Esto permite cambiar la fuente de datos sin impactar al resto del sistema.

5 Arquitectura del Sistema

5.1 Visión General

FarmaCode implementa una arquitectura distribuida de dos capas principales: un frontend Android con patrón MVVM y un backend compuesto por microservicios Java Spring Boot. La comunicación entre ambas capas se realiza mediante APIs REST sobre HTTPS. Los servicios de backend se despliegan en Railway, una plataforma cloud que gestiona el aprovisionamiento, el escalado y el ciclo de vida de los contenedores.

5.2 Arquitectura del Frontend (Android – MVVM)

La aplicación Android se organiza en capas de responsabilidad bien definidas:

- **Capa de UI (View):** pantallas implementadas con Jetpack Compose. Cada pantalla observa un ViewModel correspondiente y se recompone reactivamente ante cambios de estado.
- **Capa de ViewModel:** contiene la lógica de presentación. Gestiona el estado de la UI, invoca casos de uso del dominio y expone flujos de datos mediante `StateFlow`.
- **Capa de Dominio:** casos de uso (*Use Cases*) que encapsulan la lógica de negocio independiente del framework.
- **Capa de Datos:** repositorios que abstraen el acceso a Room (base de datos local SQLite) y al cliente HTTP (Retrofit) para consumir los microservicios.

5.3 Arquitectura del Backend (Microservicios – Spring Boot)

Cada microservicio se implementa como una aplicación Spring Boot independiente, expone su propio conjunto de endpoints REST y gestiona su propio esquema en la base de datos MySQL. Los servicios no comparten estado en memoria, garantizando la independencia de despliegue.

Capas internas de cada microservicio:

- **Controller:** capa de entrada HTTP, maneja validaciones de request y mapeo de DTOs.
- **Service:** lógica de negocio, orquestación de repositorios y llamadas a servicios externos.
- **Repository:** acceso a datos mediante Spring Data JPA / Hibernate.
- **Domain:** entidades de negocio y objetos de valor.

5.4 Flujo de una Consulta Típica

El flujo de una consulta típica en el sistema opera de la siguiente manera:

1. El usuario escanea un medicamento desde la app Android.
2. ML Kit procesa la imagen y extrae el texto del nombre del fármaco.

3. La app envía el texto al **Servicio de Consultas** en el backend.
4. Este servicio invoca la API de Gemini para normalizar el principio activo.
5. Con el principio activo identificado, se consulta al **Servicio de Medicamentos** para obtener bioequivalentes.
6. El **Servicio de Precios** adjunta el comparativo económico.
7. El resultado se devuelve a la app para su visualización.

6 Lenguaje de Programación y Stack Tecnológico

6.1 Tabla de Tecnologías

Componente	Tecnología	Rol en el Sistema
Android (Frontend)	Kotlin + Jetpack Compose	Interfaz de usuario declarativa y reactiva
Backend (Microservicios)	Java 21 + Spring Boot 3	API REST, lógica de negocio, persistencia
Base de Datos	MySQL 8	Almacenamiento relacional de fármacos y usuarios
ORM	Spring Data JPA / Hibernate	Mapeo objeto-relacional en microservicios
HTTP Client (Android)	Retrofit 2 + OkHttp	Consumo de APIs REST desde el móvil
Persistencia Local	Room (Android)	Caché offline de medicamentos consultados
OCR	ML Kit (Google)	Reconocimiento óptico de caracteres en recetas
IA / NLP	Gemini API (Google)	Normalización de principios activos
Inyección de Dependencias	Hilt (Android)	Gestión del grafo de dependencias en la app
Contenerización	Docker	Empaquetado de microservicios para despliegue

Table 2: Stack tecnológico principal del ecosistema FarmaCode.

6.2 Justificación de las Tecnologías Clave

6.2.1 Kotlin y Jetpack Compose

Kotlin es el lenguaje oficial de Google para el desarrollo Android desde 2019. Su sintaxis expresiva, su seguridad frente a nulos y su compatibilidad con corrutinas lo hacen ideal para aplicaciones móviles modernas. Jetpack Compose permite construir UIs declarativas que se recomponen eficientemente ante cambios de estado, eliminando el código boilerplate asociado a las vistas XML tradicionales.

6.2.2 Java con Spring Boot

Spring Boot es el framework de facto para el desarrollo de microservicios en el ecosistema JVM. Provee autoconfiguración, un servidor embebido (Tomcat), gestión de dependencias mediante Maven y una robusta integración con Spring Data JPA para el acceso a bases de datos relacionales. Su madurez y su amplia comunidad lo hacen una elección sólida para un backend de producción.

7 Configuración del Servidor

7.1 Railway como Plataforma Cloud

Railway es una plataforma de despliegue en la nube de tipo PaaS (*Platform as a Service*) que permite poner en producción aplicaciones y bases de datos con mínima configuración de infraestructura. Para FarmaCode, Railway aloja la totalidad del backend: cada microservicio Spring Boot se despliega como un servicio independiente dentro del mismo proyecto Railway, junto con la instancia de base de datos MySQL.

Las características de Railway aprovechadas en el proyecto son:

- **Despliegue desde GitHub:** cada push a la rama principal dispara un pipeline de CI/CD automático que reconstruye y despliega el servicio actualizado.
- **Variables de entorno seguras:** credenciales de base de datos y claves de API (Gemini) se gestionan como variables de entorno cifradas, sin exponerlas en el código fuente.
- **Base de datos MySQL administrada:** Railway provisiona y gestiona la instancia de MySQL, incluyendo backups automáticos y acceso mediante cadena de conexión JDBC.
- **Dominio público HTTPS:** cada servicio recibe un dominio público con certificado SSL automático, asegurando la comunicación cifrada con la app Android.
- **Escalado automático:** Railway escala horizontalmente los servicios ante incrementos de carga sin intervención manual.
- **Logs en tiempo real:** la plataforma expone logs de aplicación en tiempo real para facilitar el monitoreo y la depuración.

7.2 Configuración de Cada Microservicio en Railway

Cada microservicio Spring Boot se empaqueta como imagen Docker mediante un Dockerfile estándar. Railway Bura u omite configuraciones complejas detectando automáticamente el archivo en el repositorio. Las variables de entorno son:

7.3 Modelo de Servicios Cloud

- **PaaS (Platform as a Service):** Railway gestiona el runtime, el sistema operativo y el aprovisionamiento de contenedores.
- **DBaaS (Database as a Service):** la instancia MySQL es administrada por la plataforma, eliminando mantenciones manuales.
- **SaaS (Software as a Service):** la API de Gemini se consume de manera externa sin servidores dedicados a la IA.

Variable de Entorno	Descripción
SPRING_DATASOURCE_URL	URL JDBC de la base de datos MySQL en Railway
SPRING_DATASOURCE_USERNAME	Usuario de la base de datos
SPRING_DATASOURCE_PASSWORD	Contraseña (gestionada como secret)
GEMINI_API_KEY	Clave de acceso a la API de Google Gemini
SERVER_PORT	Puerto de escucha del servicio
SPRING_PROFILES_ACTIVE	Perfil activo: production

Table 3: Variables de entorno clave para el despliegue.

8 Aspectos de Integración

8.1 Integración OCR con ML Kit

La integración de reconocimiento óptico de caracteres (OCR) se implementa en la capa Android mediante **ML Kit**, la librería de aprendizaje automático on-device de Google. ML Kit procesa las imágenes capturadas por la cámara del dispositivo directamente sin necesidad de enviar datos a la nube, garantizando velocidad y privacidad en la captura.

El flujo de integración OCR es el siguiente:

1. El usuario activa el escáner desde la pantalla principal de la app.
2. La cámara captura un fotograma que es procesado por el **TextRecognizer** de ML Kit.
3. ML Kit identifica y extrae los bloques de texto detectados en la imagen (nombre del fármaco, dosis, laboratorio).
4. El texto extraído se normaliza y se envía al Servicio de Consultas del backend mediante una petición POST.
5. En paralelo, se muestra feedback visual al usuario indicando que el reconocimiento fue exitoso.

ML Kit fue elegido sobre alternativas como Tesseract por su rendimiento superior en dispositivos Android modernos, su integración nativa con el ecosistema Jetpack y su capacidad de funcionar completamente offline en el dispositivo.

8.2 Integración con la API de Gemini

Una vez que el backend recibe el texto extraído por OCR, el Servicio de Consultas invoca la **API de Gemini** de Google para realizar la normalización del principio activo. Gemini es un modelo de lenguaje multimodal de gran escala que permite interpretar nombres comerciales de fármacos y mapearlos con precisión a su denominación común internacional (DCI) y principio activo correspondiente.

El flujo de integración con Gemini es el siguiente:

1. El Servicio de Consultas construye un prompt estructurado enviando el nombre del medicamento detectado por OCR.
2. El prompt instruye a Gemini a responder exclusivamente con un objeto JSON que contiene el principio activo, la concentración y la forma farmacéutica.
3. La respuesta JSON de Gemini es parseada y validada por el servicio antes de continuar el flujo.
4. Con el principio activo identificado, el servicio consulta la base de datos de bioequivalentes y devuelve la lista de alternativas al cliente Android.

El uso de un prompt con respuesta en formato JSON estructurado garantiza que las

respuestas de Gemini sean programáticamente procesables, reduciendo la ambigüedad y simplificando el parsing en el backend.

8.3 Integración Android ↔ Backend (Retrofit)

La comunicación entre la aplicación Android y los microservicios Spring Boot se realiza mediante **Retrofit 2** con OkHttp como cliente HTTP subyacente. Retrofit permite declarar las interfaces de los endpoints como interfaces Kotlin con anotaciones, generando el código HTTP de forma automática.

Características clave de la integración:

- **Serialización:** Gson o Moshi convierten los objetos Kotlin a JSON y viceversa de forma automática.
- **Corrutinas:** las llamadas de red se realizan en corrutinas de Kotlin, evitando el bloqueo del hilo principal de la UI.
- **Manejo de errores:** se implementa un interceptor global que captura errores HTTP (4xx, 5xx) y los transforma en excepciones tipadas del dominio.
- **Certificados SSL:** todas las comunicaciones ocurren sobre HTTPS con los dominios provistos por Railway, sin configuración adicional de certificados.

9 Conceptualización de la Solución

FarmaCode representa un ecosistema tecnológico completo que conecta la experiencia del usuario en el punto de compra con una infraestructura cloud escalable e inteligencia artificial aplicada al dominio farmacéutico. La solución entrega valor en tres dimensiones:

- **Económica:** reducción del gasto de bolsillo mediante la identificación de bioequivalentes de menor costo con idéntica eficacia terapéutica.
- **Informativa:** acceso inmediato a datos farmacológicos precisos sin depender del consejo del vendedor.
- **Social:** democratización del acceso a información de salud, con impacto especial en sectores vulnerables y enfermos crónicos.

9.1 Atributos de Calidad

Atributo	Descripción
Disponibilidad	Los servicios Railway garantizan uptime con reinicio automático ante fallos.
Seguridad	Comunicaciones HTTPS, credenciales en variables de entorno cifradas, sin datos sensibles en código fuente.
Escalabilidad	Microservicios independientes que escalan horizontalmente según demanda en Railway.
Precisión	Datos farmacológicos validados contra el vademécum ISP y normalizados mediante Gemini.
Usabilidad	Interfaz Material Design 3 con flujos de usuario optimizados para el punto de venta.
Mantenibilidad	Arquitectura en capas (MVVM + Microservicios) que permite modificar componentes de forma aislada.

Table 4: Atributos de calidad evaluados en el sistema.

11 Conclusión

FarmaCode constituye una respuesta técnica sólida y fundamentada a un problema concreto de salud pública y equidad económica en el mercado farmacéutico chileno. La combinación de una metodología ágil (Scrum), una arquitectura distribuida (MVVM + Microservicios), el uso estratégico de inteligencia artificial (Gemini API) y el despliegue en una plataforma cloud moderna (Railway) conforman un sistema coherente, escalable y con potencial de crecimiento real.

Las decisiones tecnológicas adoptadas —Kotlin/Compose en el frontend, Java/Spring Boot en backend, ML Kit para OCR— no son arbitrarias: responden a criterios de madurez de las tecnologías, compatibilidad con el ecosistema cloud y productividad del equipo de desarrollo. El resultado es una aplicación que equilibra rigor técnico con impacto social tangible.

El proyecto sienta las bases para futuras extensiones: integración con el portal de precios del MINSAL, soporte para múltiples países de la región, o evolución hacia un asistente farmacológico conversacional apoyado en modelos de lenguaje de mayor capacidad.

10 Mockup

19:34 4G 51%



Crear Cuenta

 Nombre completo

 Correo electrónico

 Contraseña 

 Confirmar contraseña 

Registrarse

¿Ya tienes cuenta? [Inicia sesión](#)

Figure 1: Registro de usuario

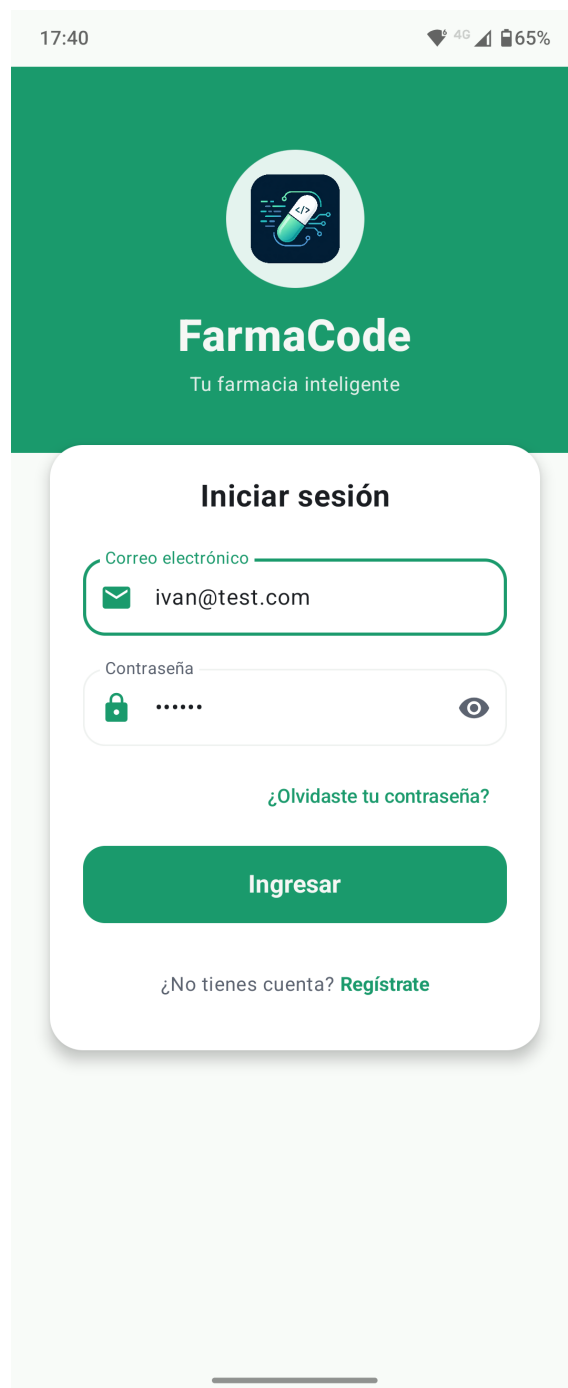


Figure 2: Inicio de sesión

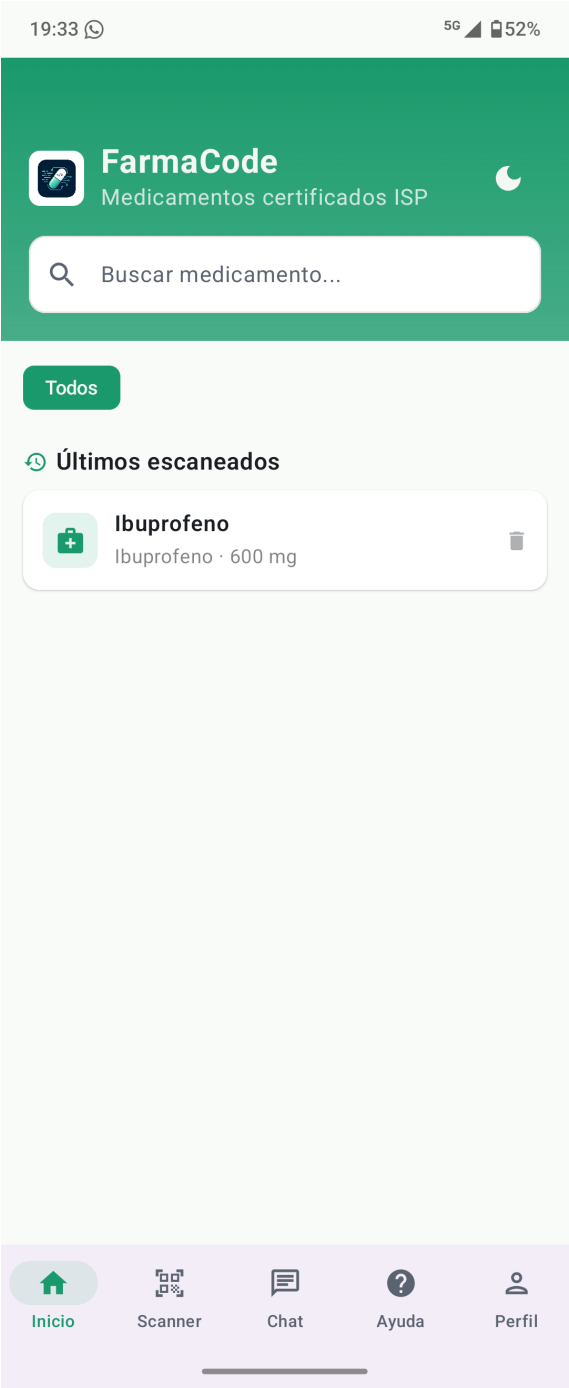


Figure 3: Inicio - Historial



Figure 4: Escáner de medicamentos

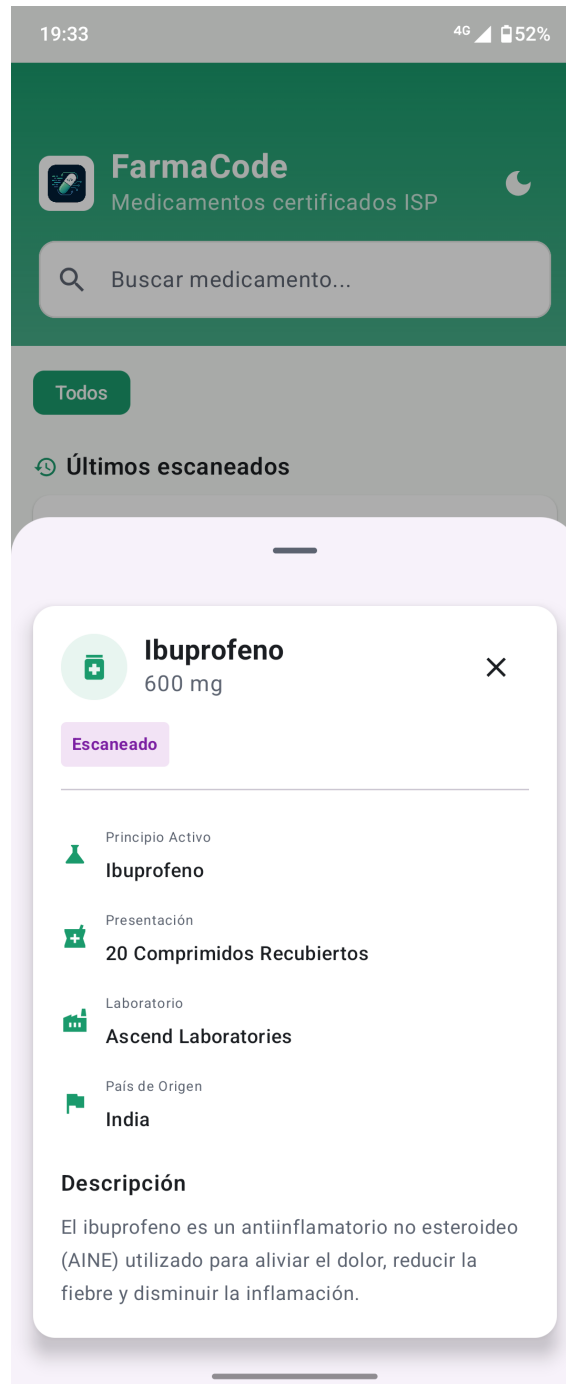


Figure 5: Resultado de análisis



Figure 6: Escáner y filtro de medicamentos

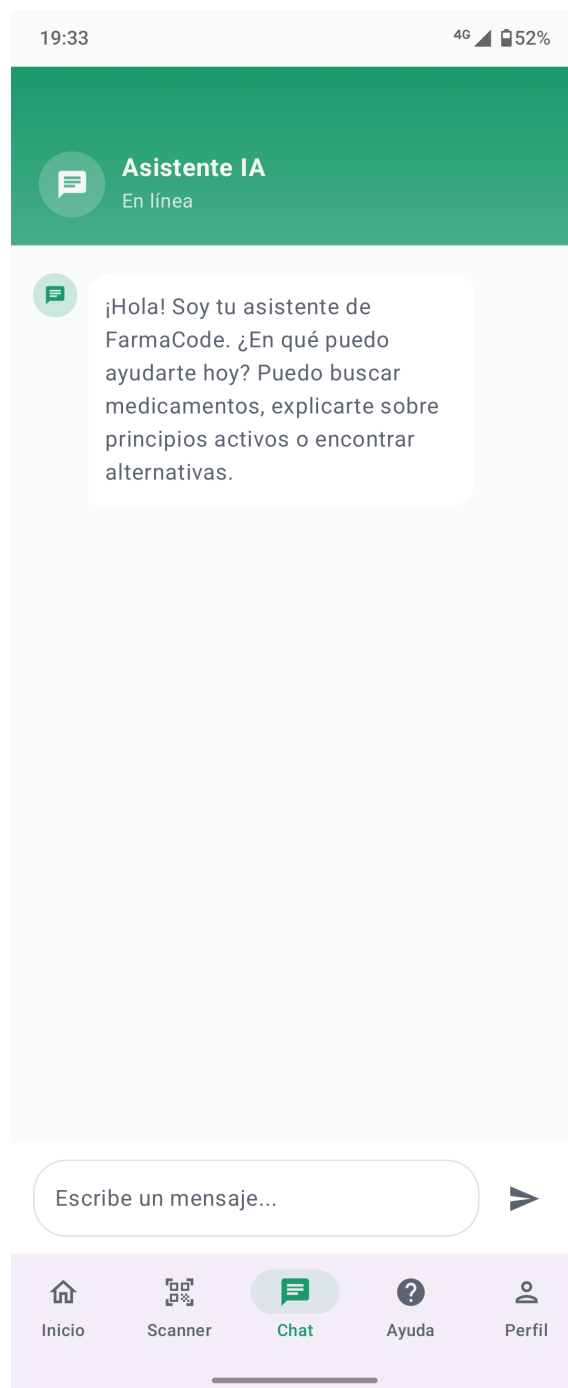


Figure 7: Asistente IA



Figure 8: Ayuda



Figure 9: Ayuda

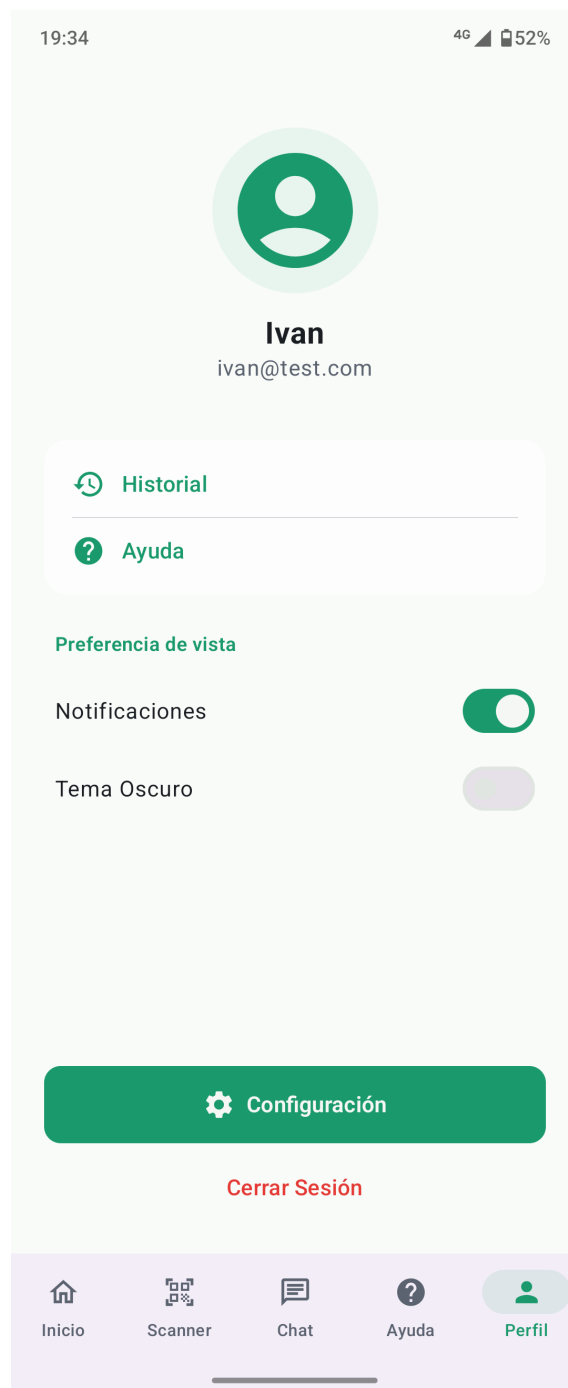


Figure 10: Perfil



Figure 11: Ajustes